بِسْمِ اللَّهِ الرَّحْمَٰنِ الرَّحِيمِ

LIVE ONLINE COURSE | COHORT FALL 2026

AI & Machine Learning for Civil Engineers

SESSION 03 · LINEAR REGRESSION, ERROR METRICS & CROSS-VALIDATION

Your First Engineering Prediction Model

Can a model learn a useful relationship from the data we prepared, and how would we know?

DATA → MODEL → PREDICTION → EVALUATION → DECISION



Dr. Abdul Waris
Research Associate · Universität Siegen, Germany

Technology partner QUDELTA STUDIOS

WHERE ARE WE? · FROM DATA PREPARATION TO PREDICTION

We have prepared the data. Now we predict.

SESSIONS 01–02: WE ALREADY LEARNED TO

Understand an engineering dataset	`data dictionary`
Identify features and target	`X, y`
Check data quality	`isna · duplicated`
Explore relationships	`hist · corr`
Avoid data leakage	`known in time?`
Split data into training and test sets	`train_test_split`

TODAY · SESSION 03

Can we use these inputs to predict an engineering response?

Build our first model, then **evaluate it honestly.**

THE COURSE AT A GLANCE

Eight sessions. One engineering workflow.

PART 1 · FOUNDATIONS

01 From Engineering Problems to Python DONE
AI/ML basics, Google Colab, NumPy & pandas

02 From Raw Engineering Data to an ML-Ready Dataset DONE
EDA, data quality, outliers, train/test split, leakage

03 Your First Engineering Prediction Model TODAY
Linear regression, error metrics, cross-validation

04 Nonlinear Engineering Problems: Trees & Ensembles
Decision trees, Random Forest, boosting

PART 2 · PREDICTION TO DECISION

05 From Prediction to Engineering Decision: Classification
Logistic regression, confusion matrix, precision & recall

06 Artificial Neural Networks
Architecture, activations, training, overfitting

07 Can We Trust Our ML Model?
Feature importance, SHAP, engineering judgement
Instructor's research area

08 From Prediction to Probability
Monte Carlo, ML surrogates, probability of failure
Instructor's research area

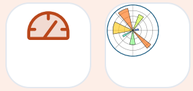
TODAY · SESSION 03

Your first engineering prediction model

01 **Build the model**
The question · a baseline · training · linear regression · the maths inside



02 **How good is it?**
MAE · RMSE · R^2 · train vs test · parity and residual plots



03 **Can we trust it?**
Split sensitivity · cross-validation · the test-set rule · engineering judgement



BY THE END OF TODAY YOU CAN

- Fit a linear-regression model and compare it with a baseline
- Explain what training, a cost function and gradient descent do
- Read MAE, RMSE and R^2, and diagnose a model with residuals
- Use cross-validation for decisions and the test set only once

01

PART 01 · BUILD THE MODEL

From engineering question to a trained model

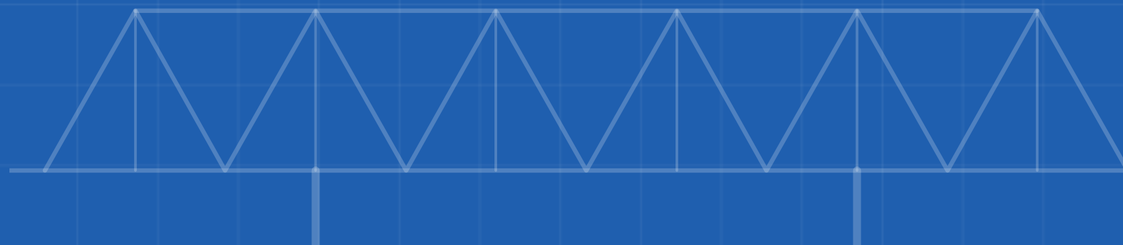
a Our engineering question

b A baseline to beat

c What “training” means

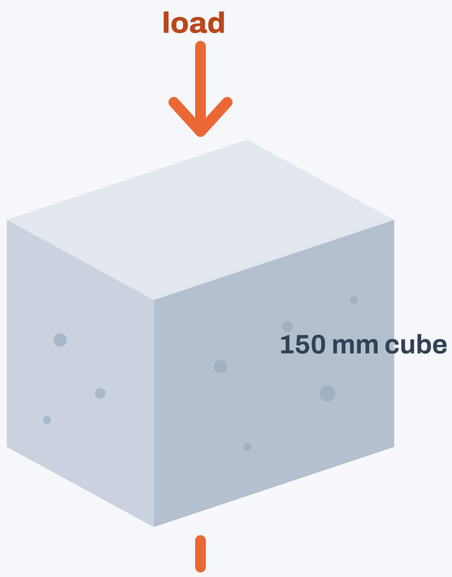
d Linear regression and its weights

e The maths: cost, gradient descent, scaling



OUR ENGINEERING QUESTION

Can we predict concrete compressive strength?



We know the mix and the curing age. Can we estimate the strength **without performing the test?**

Cement kg/m³	**Blast-furnace slag** kg/m³	**Fly ash** kg/m³	**Water** kg/m³
Superplasticizer kg/m³	**Coarse aggregate** kg/m³	**Fine aggregate** kg/m³	**Age** days

8 INPUTS → MODEL →

y **Compressive strength** MPa

“If I give you a new concrete mix, could you estimate its strength without testing it?”

THE DATASET · UCI CONCRETE COMPRESSIVE STRENGTH

Our first ML prediction dataset

1,030
concrete mixes in the file

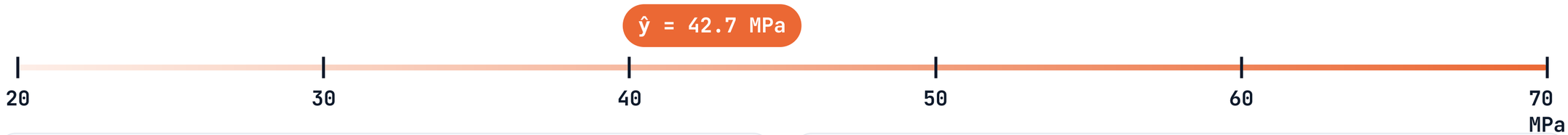
1,005
after removing the 25 duplicate rows found in Session 02

8
input variables X, no missing values

1
target y: compressive strength in MPa

THE ML PROBLEM

A regression problem: the target is a continuous quantity



Inputs · X **Mix proportions + age** — 8 numbers per mix

Target · y **Compressive strength** — any value on the line, not a category

Data: UCI Concrete Compressive Strength (Yeh, 1998). From today the notebook uses `df.drop_duplicates()` before splitting, as recommended in Session 02.

REGRESSION IN ONE PICTURE · A REAL TEST MIX

Regression = predicting a number, then checking it

THE MODEL RECEIVES · X

Cement	146.5
Slag	114.6
Fly ash	89.3
Water	201.9
Superplasticizer	8.8
Aggregates (coarse · fine)	860 · 830
Age (days)	28

Trained model

linear regression, fitted on the training mixes

Predicted ŷ	22.5 MPa
Measured y	20.0 MPa
Prediction error ŷ – y	+2.5 MPa

Every regression model produces predictions that we must compare against observations.

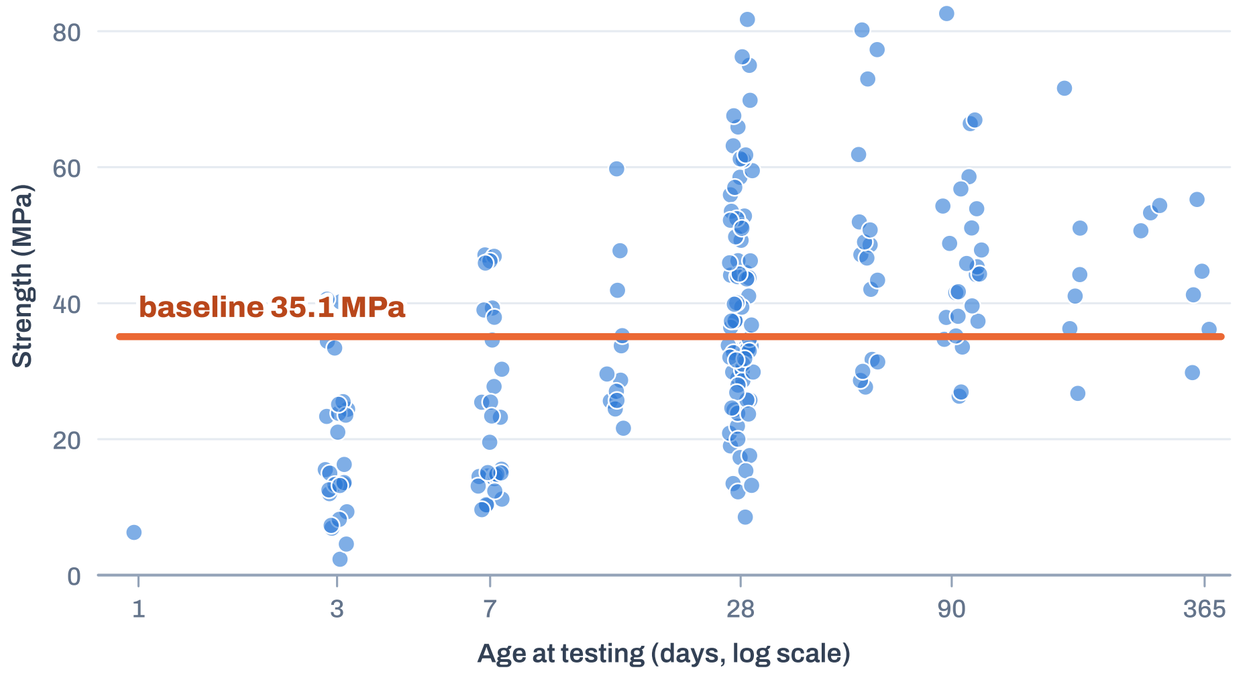
One of the 201 test mixes in the notebook (inputs in kg/m³). The model never saw it during training.

BEFORE ML: ESTABLISH A BASELINE

Is our model actually useful? Compared with what?

The mean baseline ignores all eight inputs

It predicts the average training strength for every mix. Dots: the 201 test mixes.



THE SIMPLEST PREDICTION

Average training strength = **35.1 MPa** → predicted for every concrete mix.

HOW WRONG IS IT?

On the test mixes it is off by **14.0 MPa** on average.

A RESEARCH HABIT

A useful model must beat this.

Never report "R^2 = 0.75" without saying: compared with what?

WHAT DOES “TRAINING” MEAN?

Training = learning model parameters from examples

Training examples
804 mixes with their measured strength

The model learns its parameters
values that make predictions as close as possible to the measurements

A fitted model
ready to predict a mix it has never seen

Session03_first_model.ipynb fit

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

FOR LINEAR REGRESSION THE PARAMETERS ARE

`intercept_` one number: the starting value

`coef_` one weight for each of the 8 inputs

OUR FIRST PROTECTION AGAINST OVERCONFIDENCE

Why don’t we train on everything?

Studying with the exact questions that will appear in the exam gives a high score that says little about new questions.

ML models face the same problem. So we hold data back that the model never sees while learning.

80% · Training data

804 mixes

Fitting the model | Model development | Cross-validation

20% · Test

201 mixes

Final evaluation

The test set represents new data from the model’s perspective.

LINEAR REGRESSION: OUR FIRST MODEL

Why start with linear regression?

THE PREDICTION IS A WEIGHTED SUM OF THE INPUTS

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_8 x_8$$

x_j
the input variables

β_j
learned coefficients (weights)

β_0
the intercept

$\hat{y}$
predicted strength

Simple

One equation you can write on a board.

Fast

Fits 804 mixes in milliseconds.

Interpretable

Every input has a visible weight.

A benchmark

The reference for trees and neural networks later.

Simple does not mean useless.

WHAT DOES THE MODEL LEARN? · OUR FITTED MODEL

The model learns the weights

FITTED ON 804 TRAINING MIXES

Strength = `-23.39` + `0.117` ·cement + `0.098` ·slag + `0.085` ·fly ash `-0.131` ·water + `0.331` ·superplasticizer + `0.016` ·coarse + `0.021` ·fine + `0.111` ·age

WE DON'T CHOOSE THESE NUMBERS

The algorithm estimates them from the examples we provide. This is where **the data starts writing the model,** instead of us writing the rules.

Session03_first_model.ipynb — weights

```
model.intercept_    # β0
model.coef_         # β1 … β8
```

OUTPUT

```
-23.39
[0.117, 0.098, 0.085, -0.131, 0.331, 0.016, 0.021,
0.111]
```

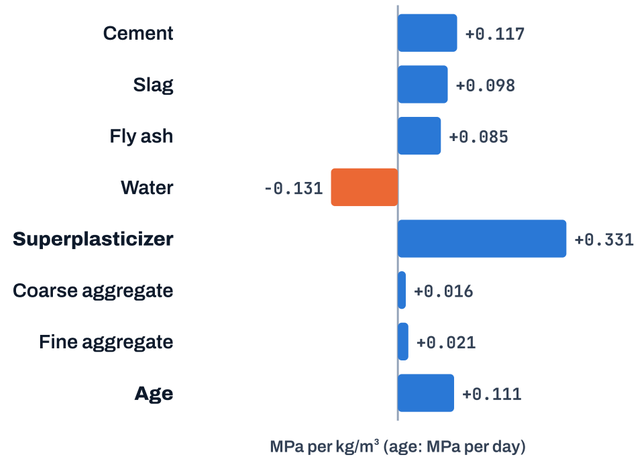
Units: MPa per kg/m³ for the mix quantities, MPa per day for age; intercept in MPa. Rounded to three decimals.

LOOKING INSIDE THE MODEL

What do the coefficients tell us?

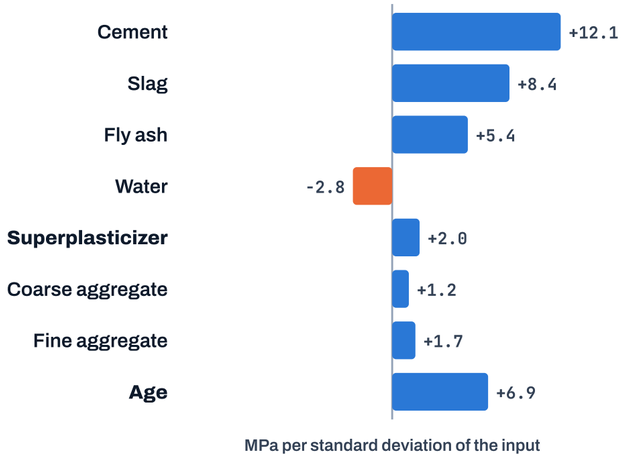
Raw coefficients

Change in predicted strength per unit of each input, others held constant.



Same model, standardised inputs

Change per one standard deviation of each input.



READING ONE

Age +0.111: within this model, one extra day adds about 0.111 MPa, holding the other inputs constant.

BUT BE CAREFUL

Units matter: superplasticizer has the largest raw weight but one of the smallest effects per standard deviation.

A larger coefficient does not automatically mean greater importance. A coefficient shows **association in the fitted model**, not causation.

This is an important engineering judgement point. More in Session 07.

SOME MATHS BEHIND ML MODELS · 1 OF 5

Linear regression: the base model of statistical ML

THE MODEL · ONE INPUT

$$f_{w,b}(x^{(i)}) = w\,x^{(i)} + b$$

Parameters: slope w and intercept b.

THE COST · SQUARED-ERROR COST FUNCTION

$$J(w,b) = \frac{1}{2m} \Sigma_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$

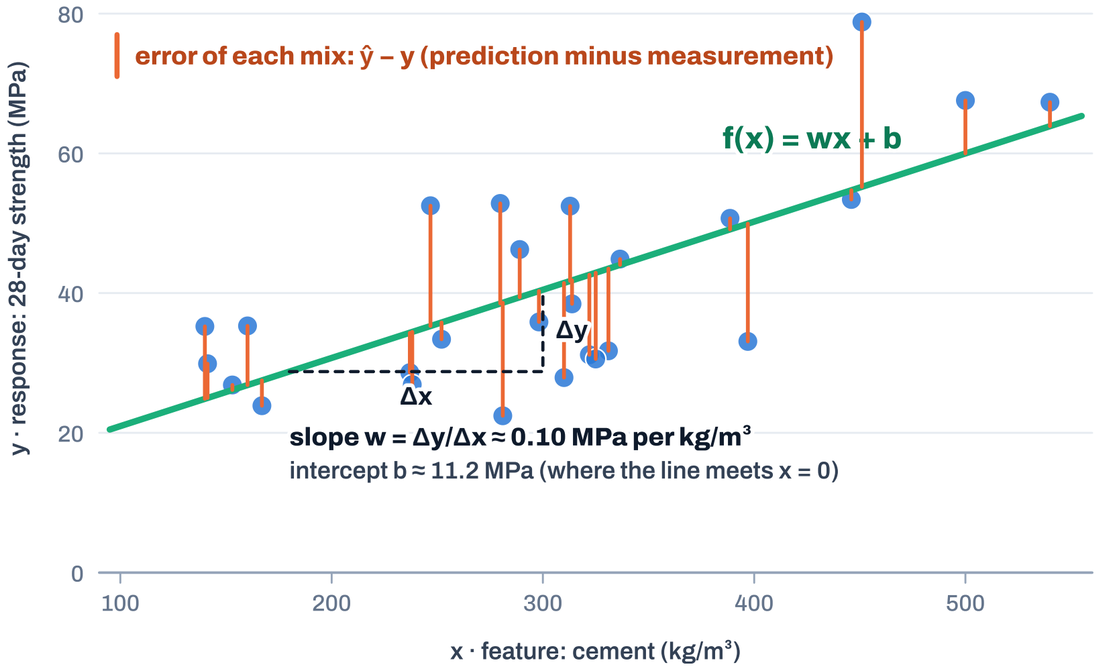
Square every error, average over the m examples.

THE GOAL

minimise $J(w, b)$ over w, b

With 8 inputs it is the same idea: one w per input.

26 real 28-day mixes, one input: cement



Notation after DeepLearning.AI (Machine Learning Specialization)

SOME MATHS BEHIND ML MODELS · 2 OF 5

The cost function is a bowl. We walk downhill.

CONVEX

For linear regression, $J(w, b)$ has a single lowest point: no false valleys.

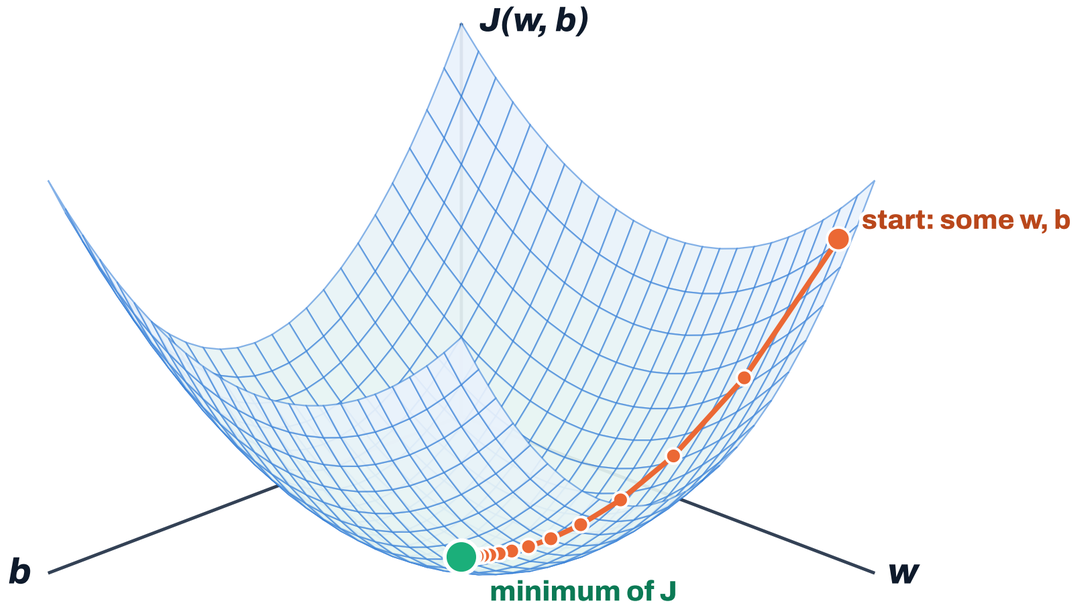
GRADIENT DESCENT, IN OUTLINE

01 Start with some w, b

02 Keep changing w, b to reduce $J(w, b)$

03 Until we settle at or near a minimum

Here: 40 steps take the cost from **1,103** to **54** MPa^2.



Cost surface computed from the 26 mixes of the previous slide (cement standardised). Concept after DeepLearning.AI.

SOME MATHS BEHIND ML MODELS · 3 OF 5

Gradient descent: follow the slope downhill

Repeat until convergence:

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

α learning rate: step size

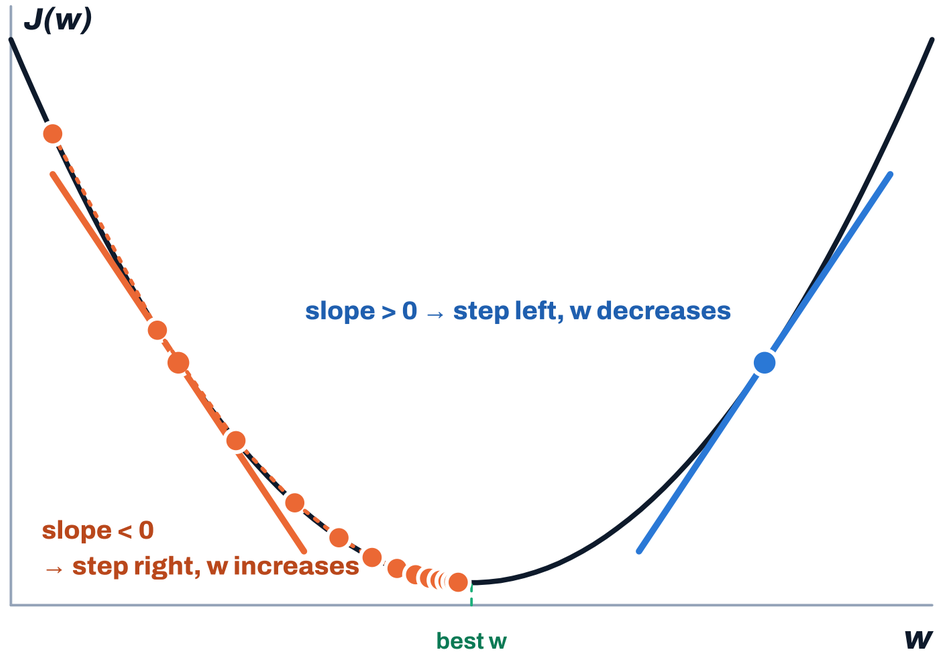
derivative: slope of J

WHY IT WORKS

A negative slope makes *w* grow; a positive slope makes it shrink. Steps get smaller as the slope flattens near the minimum.

Update *w* and *b* **together**, using the slope at the current point.

J(*w*) with *b* at its best value



Notation after DeepLearning.AI; concept also in CS771, IIT Kanpur

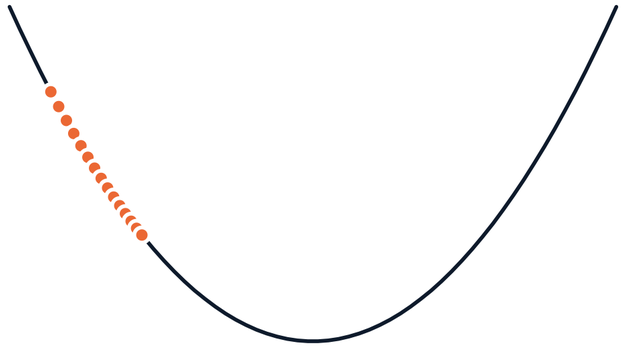
SOME MATHS BEHIND ML MODELS · 4 OF 5 · HYPERPARAMETER TUNING

The learning rate α decides whether we arrive

α = 0.03 · TOO SMALL

Slow

Tiny steps: after 14 steps we have covered only 35% of the way.

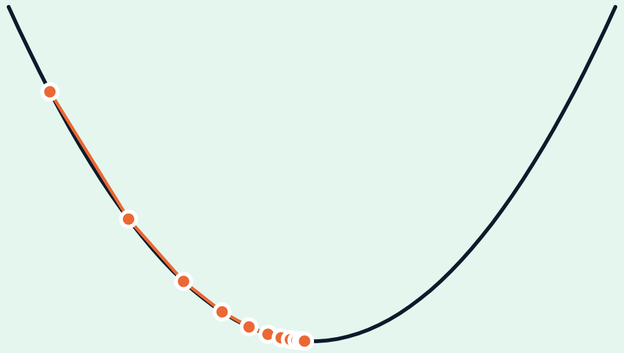


Converges, eventually

α = 0.3 · ABOUT RIGHT

Efficient

Large steps far away, small steps near the minimum: there in about 10 steps.

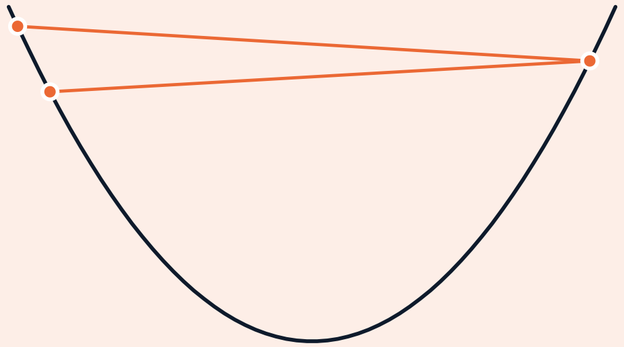


Converges quickly

α = 2.06 · TOO LARGE

Overshoots

Each step jumps past the minimum and lands higher up the other side.



Fails to converge

α is a hyperparameter: we choose it; the model does not learn it. scikit-learn's LinearRegression solves for the weights directly, so today we do not tune α.

All three runs start from the same point on the same cost curve as the previous slide.

SOME MATHS BEHIND ML MODELS · 5 OF 5 · DATA PRE-PROCESSING

Feature scaling: make the bowl round

Once the features are selected, they may not be on the same scale: cement varies by about 104 kg/m³, superplasticizer by about 6.

PROBLEMATIC

Features with larger scales can dominate comparisons, and gradient descent crawls along a long, narrow valley.

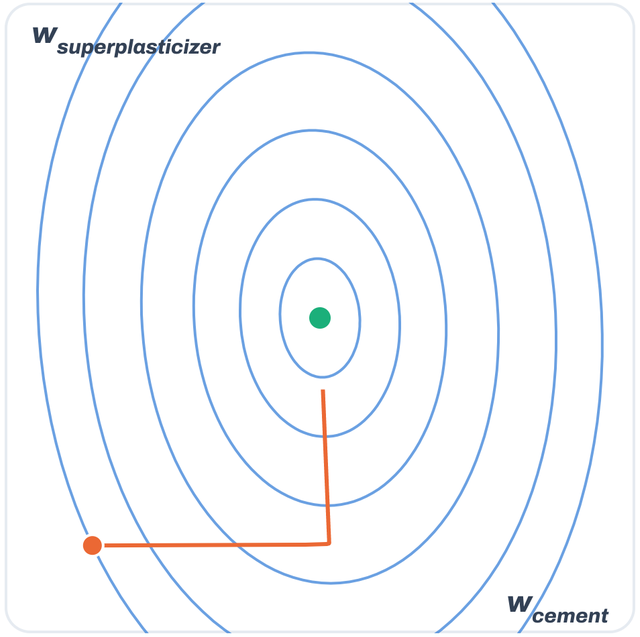
THEREFORE, STANDARDISE

Mean 0, standard deviation 1 for every feature. It stabilises the optimisation used in ML algorithms.

```
StandardScaler().fit(X_train)
```

GRADIENT DESCENT WITHOUT SCALING

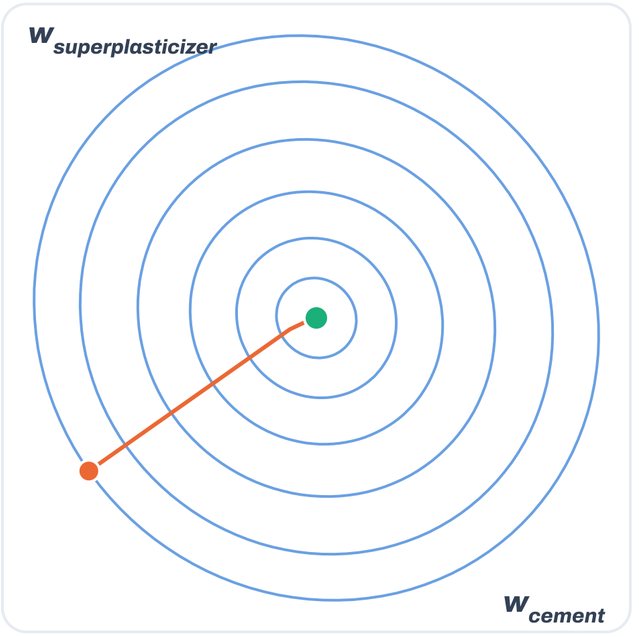
Raw kg/m³



984 iterations

GRADIENT DESCENT AFTER SCALING

Standardised



2 iterations

Real cost contours for predicting strength from cement and superplasticizer (1,005 mixes), same safe learning rate rule, stopping at 0.1% of the starting cost. Idea after Virginia Tech ECE 5424 (Lec. 03) and DeepLearning.AI.

02

a A scorecard: MAE, RMSE, R^2

b Which metric should we trust?

c Train vs test performance

d Parity plot

e Residuals: where is it wrong?

PART 02 · EVALUATION

How good is our model?

PREDICTION IS NOT ENOUGH

A model can make predictions. How good are they?

We need a scorecard. Today: three regression metrics.

MAE

Mean Absolute Error

unit: MPa · lower is better

On average, how large is the error?

RMSE

Root Mean Squared Error

unit: MPa · lower is better

How large is the error, with big misses counting more?

R^2

Coefficient of Determination

no unit · higher is better, max 1

How much better than predicting the mean?

Different metrics answer slightly different questions.

METRIC 1 · ON OUR 201 TEST MIXES

MAE: Mean Absolute Error

QUESTION · ON AVERAGE, HOW LARGE IS OUR PREDICTION ERROR?

$$\text{MAE} = \frac{1}{m}\sum_{i=1}^{m} | y_i - \hat{y}_i |$$

y_i actual target value · $\hat{y}_i$ predicted target

WHY ENGINEERS LIKE MAE

- Same units as the target: MPa
- Easy to interpret
- Every error contributes in proportion to its size

LINEAR REGRESSION · TEST SET

8.9 MPa

“Our predictions are off by about 8.9 MPa on average.”

Mean baseline: **14.0 MPa**. The model cuts the average error by 36%.

METRIC 2 · LARGE ERRORS ARE PENALISED MORE STRONGLY

RMSE: Root Mean Squared Error

SQUARES THE ERRORS BEFORE AVERAGING

$$\text{RMSE} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}$$

WHY SQUARING MATTERS · ERRORS OF TWO MODELS ON 4 MIXES

A	3	3	3	3	→ MAE 3.0 · RMSE 3.0
B	0	0	0	12	→ MAE 3.0 · RMSE 6.0

MAE → average error size

RMSE → average error with a greater penalty for large misses

LINEAR REGRESSION · TEST SET

11.2 MPa

RMSE ≥ MAE always. Here the gap (11.2 vs 8.9) says some mixes are missed by a lot.

Mean baseline: **17.3 MPa**. Lower is better.

METRIC 3 · HOW MUCH VARIATION DOES THE MODEL EXPLAIN?

R^2: compared with the mean baseline

MODEL ERROR RELATIVE TO BASELINE ERROR

$$R^2 = 1 - \frac{\Sigma (y_i - \hat{y}_i)^2}{\Sigma (y_i - \bar{y})^2}$$

$\bar{y}$ arithmetic mean of y = the baseline

$R^2 = 1$
perfect predictions

$R^2 = 0$
no better than the mean

$R^2 < 0$
worse than the mean, possible on unseen data

IMPORTANT · R^2 IS NOT ACCURACY

R^2 = 0.58 does not mean “58% of predictions are correct”. It means the squared error is 58% smaller than the mean baseline’s.

LINEAR REGRESSION · TEST SET

0.58

The model explains well over half the variation in strength that the mean leaves unexplained.

Mean baseline: **0.00**, by construction.

DON'T ASK "WHICH METRIC IS BEST?" · ASK WHAT EACH ONE TELLS US

Which metric should we trust?

METRIC	WHAT IT TELLS US	MEAN BASELINE	LINEAR REGRESSION
MAE	Typical size of the errors	14.0 MPa	8.9 MPa
RMSE	Errors, with greater emphasis on large misses	17.3 MPa	11.2 MPa
R^2	Variation explained, relative to the mean baseline	0.00	0.58

ENGINEERING JUDGEMENT

A good R^2 can still hide errors too large for the application. Is ±8.9 MPa acceptable for your design decision?

A metric is evidence, not judgement.

All values on the 201 test mixes (random_state = 42).

TRAINING PERFORMANCE ≠ GENERALISATION

Does the model work on new data?

EVALUATED ON	MAE	RMSE	R^2
Training data · seen	8.0	10.0	0.61
Test data · never seen	8.9	11.2	0.58

Our model: similar on both, and modest on both. Not overfitting; more likely **too simple** for this problem.

Similar performance
→ encouraging

Much better on training than test
→ possible overfitting / poor generalisation

Poor or modest on both
→ model may be too simple, data insufficient, or the problem difficult

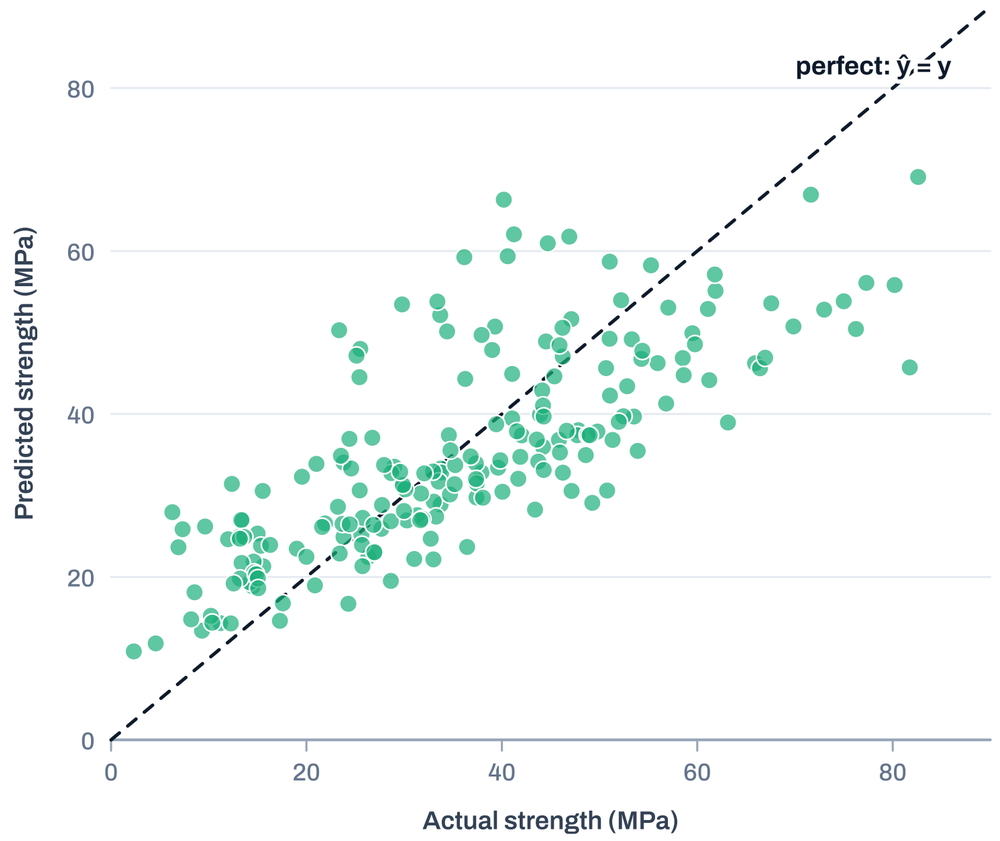
⚠ **Good training performance alone is not enough.**

MAE and RMSE in MPa. Training: 804 mixes; test: 201 mixes.

VISUALISING PREDICTIONS

Parity plot: actual vs predicted

201 test mixes



IF PREDICTIONS WERE PERFECT

Every point would lie on the 45° line.

LOOK FOR

- How close points are to the line
- Systematic deviations
- Extreme errors
- Different behaviour in different ranges

HERE

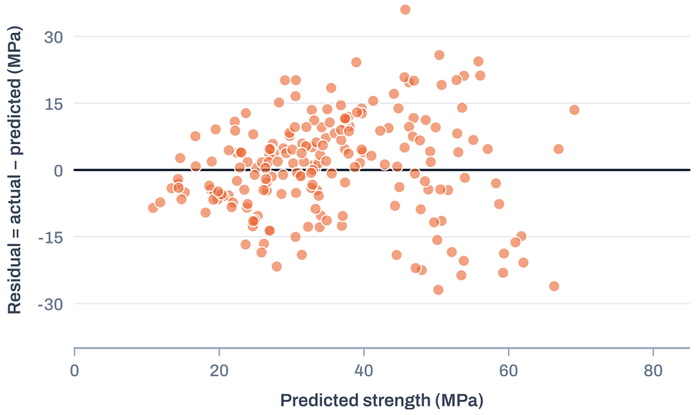
Strong mixes above 60 MPa sit below the line: the model **under-predicts high strength.**

RESIDUAL = ACTUAL – PREDICTED · WHERE AND WHY IS THE MODEL WRONG?

Residuals help us diagnose the model

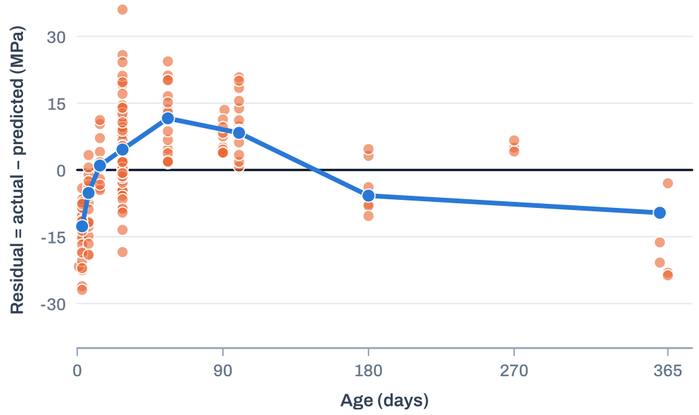
Against the prediction

Ideally: centred on zero, no pattern.



Against age: a clear curve

Blue: median residual at each test age.



WARNING SIGNS

Curvature

→ the linear model misses nonlinear behaviour. Here: –12.7 MPa at 3 days, +11.6 at 56, –9.6 at a year.

Increasing spread

→ the error changes with the target or input range

Large isolated residuals

→ unusual observations or difficult cases

Don’t just ask “how good is the score?” Ask “where and why is the model wrong?”

03

PART 03 · VALIDATION & JUDGEMENT

Is it consistent, and can we trust it?

a One split is only one split

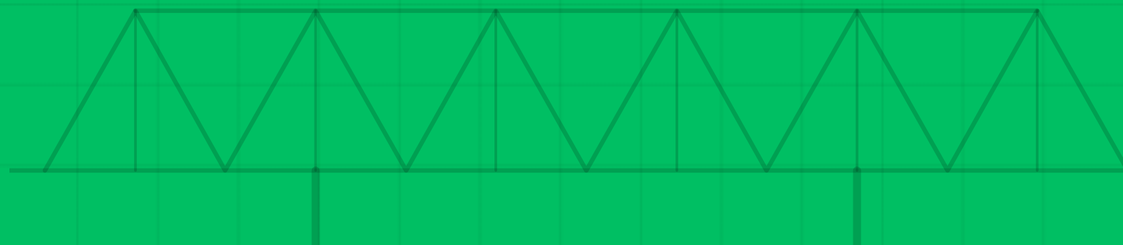
b K-fold cross-validation

c Look beyond the mean

d The test-set rule

e From performance to engineering judgement

f Your turn: diagnose the model

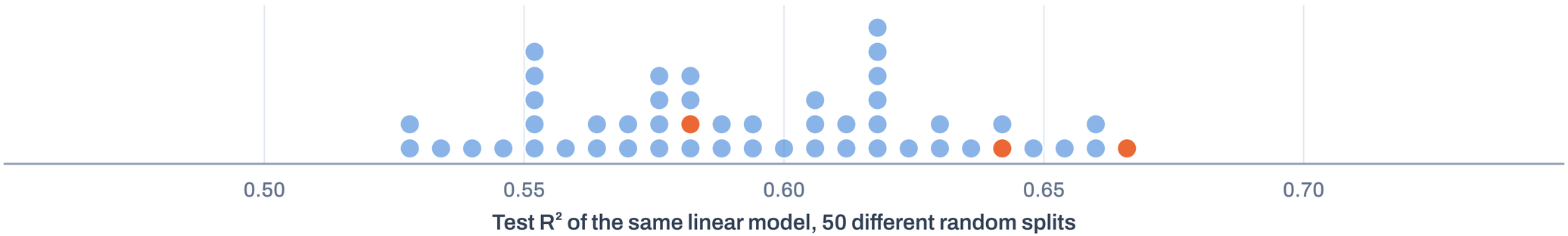


ONE SPLIT IS ONLY ONE SPLIT

What if we change the split? Same model, same data.

`random_state = 42`	R^2 0.58
`random_state = 7`	R^2 0.64
`random_state = 21`	R^2 0.66

Test R^2 from 50 different random splits: 0.53 to 0.66



Which mixes land in the test set changes the score. A single split gives one estimate of performance.

A MORE SYSTEMATIC VIEW

K-fold cross-validation

1,005 mixes → 80% training: 804 | 20% test: locked

5-FOLD CV ON THE TRAINING DATA ONLY

Fold 1	validate	train	train	train	train
Fold 2	train	validate	train	train	train
Fold 3	train	train	validate	train	train
Fold 4	train	train	train	validate	train
Fold 5	train	train	train	train	validate

Average performance metric = $(1/K)\ \Sigma\ score_k$

EACH MIX GETS ONE TURN

Every training observation is validated exactly once, by a model that did not see it.

Session03_first_model.ipynb — cv

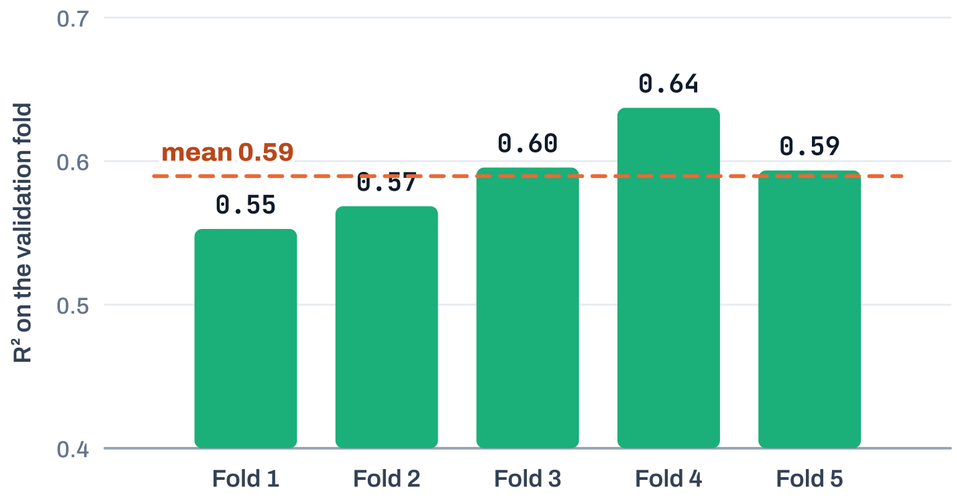
```
from sklearn.model_selection import \
    cross_val_score, KFold
cv = KFold(5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_train,
    y_train, cv=cv, scoring="r2")
```

WHAT DOES CROSS-VALIDATION TELL US? · LOOK BEYOND THE MEAN

Average and spread both matter

Our linear regression: R^2 0.59 ± 0.03

Five validation folds of the training data.



An illustration with two hypothetical models:

Model A — mean ≈ 0.90 · consistent

0.89	0.91	0.90	0.92	0.88

Model B — mean ≈ 0.90 · much more variable

0.60	0.97	0.91	1.00	1.02*

* An R^2 above 1 is impossible: a value like this signals a bug in the evaluation.

Similar average, much larger fold-to-fold variation: Model B deserves more investigation.

THE MOST IMPORTANT RULE FROM TODAY

Keep the test set untouched during development



The test set is for the final evaluation.

Use it once, at the end.

Do not use test performance to decide which

- **Model**
- **Features**
- **Hyperparameters**
- **Pre-processing approach**

is better. Every peek turns the test set into training data.

SEPARATE DEVELOPMENT FROM FINAL ASSESSMENT

The better workflow

01 Training data

↓

02 Cross-validation

↓

03 Model development and comparison

↓

04 Choose the final approach

↓

05 Train the final model on all training data

↓

06 Evaluate once on the untouched test set

↓

07 Report the final test performance

DECISIONS

Use cross-validation to make decisions.

ASSESSMENT

Use the test set to make the final assessment.

This becomes increasingly important when we compare several algorithms in Sessions 04–06.

FROM MODEL PERFORMANCE TO ENGINEERING JUDGEMENT

Accuracy is only part of the story

1

Does it generalise?

Was it evaluated on unseen data?

Test R^2 0.58, CV 0.59

2

Is the error acceptable?

What do MAE and RMSE mean in engineering terms?

±8.9 MPa on average

3

Physically plausible?

Does the model produce sensible results?

Straight-line age effect: no

4

Where does it fail?

Are errors concentrated in certain regions?

Early ages, high strength

5

Are we extrapolating?

Is the new mix outside the training range?

Check against df.describe()

Machine learning gives us a model. Engineering judgement tells us whether to trust it.

LIVE EXERCISE · IN THE NOTEBOOK

Your turn: diagnose the model

A

Change the random seed

Does the test performance change? By how much?

```
random_state = 7, 21, …
```

B

Baseline vs model

Did linear regression actually improve on the mean prediction? On which metric, by how much?

```
DummyRegressor()
```

C

Inspect the residuals

- Centred around zero?
- Any curvature?
- Does the spread change?
- Unusually large residuals?

```
plt.scatter(X_test.age, res)
```

ENGINEERING QUESTION **Is there evidence that our linear model is too simple for this problem?**

WHAT WE LEARNED TODAY · FROM ENGINEERING DATA TO A VALIDATED PREDICTION

The complete first modelling workflow

01 **Engineering question**

02 **Features + target**

03 **Train / test split**

04 **Baseline**

05 **Linear regression**

06 **Predictions**

07 **MAE · RMSE · R^2**

08 **Residual diagnostics**

09 **Split sensitivity**

10 **Cross-validation**

11 **Final test assessment**

12 **Engineering judgement**

THE CENTRAL LESSON

A prediction is not enough.

We need to know how well it works, how consistently it works, and whether the result makes engineering sense.

WHERE DO WE GO NEXT?

What if the relationship is not linear?

TODAY'S EVIDENCE

Linear regression assumes a simple, straight-line relationship. Our residuals curve with age. Civil engineering problems are often:

- Nonlinear
- Interactive
- Threshold-dependent
- Highly coupled

So what happens when a straight-line model is not enough?

NEXT · SESSION 04

Trees & ensemble models

Decision trees → Random Forest → Boosting

Same engineering problem. Same data.

A different modelling approach.

Compared properly, with today's workflow.

BEFORE SESSION 04

Three small things to try

1

Ten seeds

Repeat the split with ten random seeds. Record the test MAE each time: what is the range?

```
for s in range(10): …
```

2

Add log(age)

Add a feature `log(age)`. Does cross-validated R^2 improve? Do the residuals still curve?

```
np.log(df["age"])
```

3

Your acceptable error

For one decision in your field, write the error you could accept. Compare it with today's MAE.

```
MAE ≤ ? MPa
```

04 **Nonlinear Engineering Problems: Trees & Ensembles**

Decision trees, Random Forest and boosting on the same data.

Questions

Unmute, or drop your question in the chat.



Linear regression

Gradient descent

Metrics & residuals

Cross-validation

Dr. Abdul Waris
Research Associate · Universität Siegen, Germany

END OF SESSION 03

Thank you.

See you in session 04: **Nonlinear Engineering Problems: Trees & Ensembles.**

Dr. Abdul Waris
Research Associate · Universität Siegen, Germany

Course page
qudeltastudios.com/courses

QUDELTA: STUDIOS | Technology partner

APPENDIX · FOR THE INSTRUCTOR

Recommended timing for the 90-minute session

PART	SLIDES	APPROX. TIME
Problem, dataset, baseline, training, linear regression	1–14	12 min
Notebook load · drop duplicates · split · baseline · fit	—	15 min
The maths: cost function, gradient descent, learning rate, scaling	15–19	10 min
Metrics and train vs test	21–26	8 min
Notebook predictions · MAE · RMSE · R^2 · parity and residual plots	27–28	15 min
Split sensitivity, cross-validation, test-set rule, workflow	30–34	10 min
Notebook cross-validation · live exercise	36	12 min
Engineering judgement, summary, next session, questions	35, 37–41	8 min
Total		90 min